

Ochrona i Bezpieczeństwo Systemu Operacyjnego

Analiza mechanizmów ochrony, kontroli dostępu do zasobów oraz fundamentów architektury bezpiecznych systemów komputerowych.

Cele i Zadania Bezpieczeństwa SO

- **Poufność (Confidentiality)**

Zapewnienie, że dane są dostępne wyłącznie dla uprawnionych użytkowników i procesów. Mechanizmy te chronią przed nieautoryzowanym odczytem informacji wrażliwych.

- **Integralność (Integrity)**

Ochrona przed nieautoryzowaną modyfikacją lub usunięciem danych. System musi gwarantować, że informacje pozostają spójne, dokładne i wiarygodne w całym cyklu życia.

- **Dostępność (Availability)**

Gwarancja, że zasoby systemowe i usługi są dostępne dla uprawnionych podmiotów w momencie, gdy są potrzebne. Obejmuje odporność na ataki typu DoS oraz awarie sprzętowe.

Ochrona a Bezpieczeństwo – Rozróżnienie

Ochrona (Protection)

Mechanizmy wewnętrzne systemu operacyjnego, które kontrolują dostęp programów, procesów i użytkowników do zasobów zdefiniowanych przez system komputerowy.

Przykład: Separacja pamięci procesów, poziomy przywilejów procesora (pierścienie).

Bezpieczeństwo (Security)

Szersze pojęcie obejmujące obronę systemu przed zagrożeniami zewnętrznymi i wewnętrznymi, takimi jak złośliwe oprogramowanie, włamania czy błędy ludzkie.

Przykład: Uwierzytelnianie dwuskładnikowe (MFA), szyfrowanie dysków, zapory ogniowe.

Podstawowe Pojęcia Kontroli Dostępu



Podmiot (Subject)

Aktywna jednostka w systemie, taka jak użytkownik, proces lub program, która żąda dostępu do określonego zasobu w celu wykonania operacji.



Obiekt (Object)

Bierna jednostka zawierająca informację lub zasób sprzętowy, do której dostęp jest kontrolowany (np. pliki, katalogi, porty, segmenty pamięci).



Prawo dostępu

Określona operacja, którą podmiot może wykonać na obiekcie. Klasyczny zestaw to: odczyt (read), zapis (write) oraz wykonanie (execute).

Architektura Ochrony: Pierścienie (Rings)

Hierarchia Przywilejów

Nowoczesne procesory implementują sprzętowe wsparcie dla ochrony poprzez tzw. pierścienie ochrony (Protection Rings), które izolują jądro systemu od aplikacji.

-  **Ring 0 (Kernel Mode):**
Najwyższe uprawnienia. Bezpośredni dostęp do sprzętu i pamięci fizycznej.
-  **Ring 1 & 2:**
Zarezerwowane dla sterowników i usług systemowych (rzadziej używane).
-  **Ring 3 (User Mode):**
Najniższe uprawnienia. Aplikacje użytkownika działają w izolacji.

Kluczowe mechanizmy sprzętowe:

Izolacja Pamięci: Rejestry graniczne (Base/Limit) zapobiegają dostępowi do pamięci innych procesów.

Instrukcje Uprzywilejowane: Tylko kod w Ring 0 może wykonywać operacje krytyczne dla stabilności.

Uznaniowa Kontrola Dostępu (DAC)

Model **Discretionary Access Control (DAC)** opiera się na decyzjach podejmowanych przez właściciela zasobu. Jest to najczęściej stosowany model w domowych systemach operacyjnych.

Cechy charakterystyczne:

Właściciel obiektu ma pełną kontrolę nad prawami dostępu do niego.

Możliwość delegowania uprawnień innym użytkownikom.

Wysoka elastyczność i łatwość zarządzania w małych grupach.

Ryzyko: Programy działające w imieniu użytkownika dziedziczą jego pełne uprawnienia (podatność na konie

Przykład: Systemy UNIX/Linux

Klasyczne uprawnienia plików (rwx) przypisane do Użytkownika, Grupy i Pozostałych.

```
$ ls -l dokument.txt  
-rw-r--r-- 1 admin users 1024
```

Właściciel (admin) może czytać i pisać, pozostali tylko czytać.

Obowiązkowa Kontrola Dostępu (MAC)

Mandatory Access Control (MAC) to model, w którym decyzje o dostępie są podejmowane przez system na podstawie centralnej polityki bezpieczeństwa, a użytkownicy nie mogą ich zmieniać.

Etykiety Bezpieczeństwa

Każdy podmiot i obiekt posiada etykietę (np. Jawne, Poufne, Tajne). Dostęp jest przyznawany tylko wtedy, gdy poziom zaufania podmiotu odpowiada wymaganiom obiektu.

Modele Formalne

Bell-LaPadula: Skupia się na poufności ("No read up", "No write down").

Biba: Skupia się na integralności ("No read down", "No write up").

Zastosowania

Systemy wojskowe, rządowe oraz rozszerzenia bezpieczeństwa takie jak **SELinux** (Security-Enhanced Linux) czy **AppArmor**.

Kontrola Dostępu Oparta na Rolach (RBAC)

W modelu **Role-Based Access Control (RBAC)** uprawnienia nie są przypisywane bezpośrednio do użytkowników, lecz do abstrakcyjnych **ról** pełnionych w organizacji.

-  **Skalowalność:**
Znacznie łatwiejsze zarządzanie w dużych przedsiębiorstwach.
-  **Zasada minimalnych uprawnień:**
Użytkownik otrzymuje dostęp tylko do ról niezbędnych do pracy.
-  **Łatwa rotacja personelu:**
Przy zmianie stanowiska wystarczy zmiana przypisanej roli.

Przykład hierarchii uprawnień:

Rola	Uprawnienia do plików
Księgowy	Odczyt faktur, Zapis płatności
Kierownik	Odczyt raportów, Zatwierdzanie
Administrator	Pełna kontrola nad strukturą

Porównanie Głównych Modelów Kontroli

Kryterium	DAC (Uznaniowa)	MAC (Obowiązkowa)	RBAC (Rolowa)
Decydent	Właściciel obiektu	System (Administrator)	Organizacja (Role)
Elastyczność	Bardzo wysoka	Niska (sztywna)	Umiarkowana
Bezpieczeństwo	Podstawowe	Bardzo wysokie	Wysokie
Złożoność	Niska	Wysoka	Średnia
Zastosowanie	Systemy osobiste, Windows	Wojsko, systemy krytyczne	Przedsiębiorstwa, ERP

Zaawansowane Mechanizmy Ochrony SO



ASLR

Address Space Layout Randomization – losowe rozmieszczenie danych w pamięci, co utrudnia ataki typu Buffer Overflow.



DEP / NX Bit

Data Execution Prevention – oznaczanie obszarów pamięci z danymi jako niewykonalne, co zapobiega uruchomieniu złośliwego kodu.



Sandboxing

Izolowanie aplikacji w "piaskownicy", ograniczając jej dostęp do reszty systemu operacyjnego i plików użytkownika.

Wniosek: Współczesne systemy (Windows 11, macOS, Linux) stosują podejście *Defense in Depth*, łącząc wiele warstw ochrony sprzętowej i programowej.

Skuteczność Mechanizmów Autoryzacji



Wykres przedstawia szacunkowy poziom odporności na typowe ataki przejęcia konta (Phishing, Brute-force, Credential Stuffing).

Podsumowanie i Najlepsze Praktyki

Kluczowe Zasady

- ✓ **Principle of Least Privilege:**

Minimalizacja uprawnień do niezbędnego poziomu.

- ✓ **Default Deny:**

Wszystko, co nie jest jawnie dozwolone, jest zabronione.

- ✓ **Auditing:**

Regularne monitorowanie i logowanie zdarzeń systemowych.